

| Aspect          | Polymorphism                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 1.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Year            | 1996                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Feature         | Polymorphism                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| UniqueClassName | Polymorphism                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Definition      | <b>Polymorphism</b> means “many forms.” In Java, it allows the same method call or interface/reference type to behave differently depending on the actual object involved.                                                                                                                                                                                                                                                                                                                 |
| Main purpose    | To write flexible, extensible, and loosely coupled code.                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Types in Java   | <b>Compile-time polymorphism</b> — method overloading; <b>Run-time polymorphism</b> — method overriding/dynamic method dispatch.                                                                                                                                                                                                                                                                                                                                                           |
| Example         | Animal a = new Dog(); a.sound(); — the reference is Animal, but the overridden Dog.sound() executes at runtime.                                                                                                                                                                                                                                                                                                                                                                            |
| Advantages      | <ol style="list-style-type: none"> <li>1. Improves flexibility and extensibility.</li> <li>2. Supports loose coupling.</li> <li>3. Enables programming to interfaces/supertypes.</li> <li>4. Reduces duplicate code.</li> <li>5. Makes it easier to replace implementations.</li> <li>6. Supports plug-in architecture and framework design.</li> <li>7. Makes testing easier through interchangeable implementations.</li> </ol>                                                          |
| Disadvantages   | <ol style="list-style-type: none"> <li>1. Can make code harder for beginners to understand.</li> <li>2. Runtime polymorphism can make method execution less obvious.</li> <li>3. Excessive inheritance/overriding can create complicated class hierarchies.</li> <li>4. Debugging can require tracing the actual runtime type.</li> <li>5. Overuse may introduce unnecessary abstraction.</li> <li>6. Poorly designed polymorphic hierarchies can become difficult to maintain.</li> </ol> |
| When to use     | Use when multiple classes represent different implementations of the same conceptual operation or contract. It is particularly useful when the implementation may vary while the calling code should remain unchanged.                                                                                                                                                                                                                                                                     |
| When not to use | Avoid it when there is only one implementation, when the abstraction adds no meaningful value, or when inheritance is being used merely for code reuse without a genuine “is-a” relationship.                                                                                                                                                                                                                                                                                              |

|                                                  |                                                                                                                                                                                                                         |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                  |                                                                                                                                                                                                                         |
| <b>Real-time use case 1 — Payment</b>            | Payment payment = new CreditCardPayment(); payment.pay(); or new UpiPayment(). The application can process different payment methods through the same Payment contract.                                                 |
| <b>Real-time use case 2 — Database</b>           | Connection/repository abstractions can allow MySQL, PostgreSQL, Oracle, or mock implementations to be substituted without changing business logic.                                                                      |
| <b>Real-time use case 3 — Notifications</b>      | Notification n = new EmailNotification(); n.send(); can be replaced with SMS, push, or WhatsApp-style implementations while the caller uses the same contract.                                                          |
| <b>Real-time use case 4 — Collections</b>        | Code can work with List while the actual implementation is ArrayList or LinkedList: List<String> names = new ArrayList<>();.                                                                                            |
| <b>Real-time use case 5 — Web applications</b>   | A service can depend on an interface while different implementations are selected for production, testing, or different business requirements.                                                                          |
| <b>Real-time use case 6 — GUI/event handling</b> | Different event-handler implementations can respond to the same interface-defined operation.                                                                                                                            |
| <b>Simple Java example</b>                       | <pre> class Animal { void sound() {     System.out.println("Animal sound"); } } class Dog extends Animal { @Override void sound() {     System.out.println("Bark"); } } Animal a = new Dog(); a.sound(); // Bark </pre> |
| <b>Why the example is polymorphic</b>            | a is declared as Animal, but it refers to a Dog object. Java dynamically selects the overridden Dog.sound() implementation at runtime.                                                                                  |
| <b>Alternate feature — Interfaces</b>            | <b>Interfaces</b> are generally the best alternative/complement when the goal is to achieve polymorphism without requiring classes to share an inheritance hierarchy. Example: Payment p = new UpiPayment();.           |
| <b>Alternate feature — Composition</b>           | <b>Composition/delegation</b> can replace inheritance-based polymorphism when behavior should be assembled from objects rather than obtained through a class hierarchy.                                                 |
| <b>Alternate feature — Method overloading</b>    | For compile-time polymorphism, <b>method overloading</b> provides multiple methods with the same name but different parameter lists.                                                                                    |
| <b>Best practice</b>                             | Prefer <b>interfaces or suitable abstractions</b> for contracts, use inheritance when there is a genuine subtype relationship, and avoid creating polymorphic hierarchies solely for code reuse.                        |

|  |  |
|--|--|
|  |  |
|--|--|

| Aspect                    | Abstract Classes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version              | Java 1.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Year                      | 1996                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Feature                   | Abstract Classes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| UniqueClassName           | AbstractClasses                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Definition                | An <b>abstract class</b> is a class declared with the abstract keyword that cannot be instantiated directly. It can contain <b>abstract methods</b> (without implementation) as well as <b>concrete methods</b> (with implementation), fields, constructors, static methods, and other members.                                                                                                                                                                                                                                |
| Main purpose              | To provide a <b>common base class</b> containing shared state/behavior while allowing subclasses to provide their own implementations for abstract operations.                                                                                                                                                                                                                                                                                                                                                                 |
| Syntax                    | <pre>abstract class Vehicle {     abstract void start();     void stop() {         System.out.println("Stopped");     } }</pre>                                                                                                                                                                                                                                                                                                                                                                                                |
| Instantiation             | ✗ Cannot directly create an object using new AbstractClass().                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Inheritance               | A concrete subclass must implement all inherited abstract methods unless the subclass is also abstract.                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Polymorphism relationship | Abstract classes are commonly used together with <b>runtime polymorphism</b> :<br>Vehicle v = new Car(); v.start();                                                                                                                                                                                                                                                                                                                                                                                                            |
| Advantages                | <ol style="list-style-type: none"> <li>1. Provides a common abstraction for related classes.</li> <li>2. Allows both abstract and concrete methods.</li> <li>3. Promotes code reuse through shared implementations.</li> <li>4. Can contain instance variables/state.</li> <li>5. Can have constructors for initializing common state.</li> <li>6. Supports runtime polymorphism.</li> <li>7. Helps enforce required behavior in subclasses.</li> <li>8. Can provide protected helper methods shared by subclasses.</li> </ol> |
| Disadvantages             | <ol style="list-style-type: none"> <li>1. Java allows a class to extend only <b>one</b> class, limiting multiple inheritance of implementation.</li> <li>2. Creates stronger coupling between the base class and subclasses.</li> </ol>                                                                                                                                                                                                                                                                                        |

|                                                |                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                | <p>3. Changes to the abstract base class can affect many subclasses.</p> <p>4. Can become difficult to maintain if the hierarchy grows too large.</p> <p>5. Not ideal when unrelated classes need the same contract.</p> <p>6. May encourage inheritance where composition would be more appropriate.</p> <p>7. Abstract classes cannot be instantiated directly.</p> |
| <b>When to use</b>                             | Use when several closely related classes share <b>common state, implementation, and behavior</b> , but some operations must be implemented differently by each subclass.                                                                                                                                                                                              |
| <b>When not to use</b>                         | Avoid when classes are not genuinely related, when you only need a contract without shared implementation/state, when multiple unrelated classes need the same capability, or when composition provides a cleaner design.                                                                                                                                             |
| <b>Real-time use case 1 — Payment</b>          | An abstract Payment class can contain common payment validation/logging while subclasses such as CreditCardPayment, UPIPayment, and NetBankingPayment implement processPayment().                                                                                                                                                                                     |
| <b>Real-time use case 2 — Vehicle</b>          | An abstract Vehicle class can contain common properties such as speed and methods such as stop(), while Car and Bike implement start() differently.                                                                                                                                                                                                                   |
| <b>Real-time use case 3 — Employee</b>         | An abstract Employee class can contain common employee information such as name and employeeId, while subclasses implement role-specific operations.                                                                                                                                                                                                                  |
| <b>Real-time use case 4 — Banking</b>          | An abstract BankAccount can provide common balance/deposit functionality while SavingsAccount and CurrentAccount implement account-specific withdrawal or interest rules.                                                                                                                                                                                             |
| <b>Real-time use case 5 — File processing</b>  | An abstract FileProcessor can provide common file-reading/error-handling logic while subclasses implement processing for CSV, JSON, XML, etc.                                                                                                                                                                                                                         |
| <b>Real-time use case 6 — Framework design</b> | Frameworks can provide an abstract base class containing a standard workflow while application-specific subclasses customize particular steps.                                                                                                                                                                                                                        |
| <b>Java example</b>                            | <pre> abstract class Vehicle {     abstract void start();     void stop() { System.out.println("Vehicle stopped"); } }  class Car extends Vehicle {     @Override </pre>                                                                                                                                                                                              |

|                                                |                                                                                                                                                                                                                                               |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                | <pre> void start() { System.out.println("Car starts with key"); } }  Vehicle v = new Car(); v.start(); v.stop(); </pre>                                                                                                                       |
| <b>What happens in the example</b>             | Vehicle defines the common abstraction. start() is mandatory for subclasses, while stop() is already implemented. Car supplies its own implementation of start().                                                                             |
| <b>Alternate feature — Interface</b>           | <b>Interfaces</b> are the primary alternative when you need to define a contract rather than share substantial state or implementation. A class can implement multiple interfaces.                                                            |
| <b>Alternate feature — Composition</b>         | <b>Composition</b> is an alternative when behavior should be assembled from independent objects instead of inherited from a base class. It generally provides looser coupling.                                                                |
| <b>Alternate feature — Concrete base class</b> | A normal concrete superclass can be used when there is no requirement to prevent direct instantiation and subclasses can use inherited concrete behavior directly.                                                                            |
| <b>Abstract Class vs Interface</b>             | <b>Abstract class:</b> better when closely related classes need shared state, constructors, and implementation. <b>Interface:</b> better when unrelated classes need to follow the same contract or when a class needs multiple capabilities. |
| <b>Best practice</b>                           | Use an abstract class when there is a strong <b>“is-a” relationship plus meaningful shared implementation/state</b> . Otherwise, consider an interface or composition.                                                                        |

| Aspect          | Interfaces                                                                                                                                                                                        |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 1.0                                                                                                                                                                                          |
| Year            | 1996                                                                                                                                                                                              |
| Feature         | Interfaces                                                                                                                                                                                        |
| UniqueClassName | Interfaces                                                                                                                                                                                        |
| Definition      | An <b>interface</b> is a reference type that defines a contract that implementing classes agree to follow. In Java 1.0, interfaces primarily declared abstract methods and constants; modern Java |

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            | has expanded interfaces to also support default, static, and private methods.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Main purpose</b>        | To define <b>what a class must do</b> without requiring it to inherit implementation from a particular class. Interfaces are fundamental to abstraction, polymorphism, loose coupling, and multiple-type inheritance.                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Basic syntax</b>        | <pre>interface Payment { void pay(double amount); } class UpiPayment implements Payment { public void pay(double amount) { /* implementation */ } }</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Implementation</b>      | A class uses the implements keyword to implement an interface.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Multiple interfaces</b> | A Java class can implement multiple interfaces: class SmartPhone implements Camera, GPS, MusicPlayer.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Advantages</b>          | <ol style="list-style-type: none"> <li>1. Promotes loose coupling.</li> <li>2. Defines a clear contract between components.</li> <li>3. Supports runtime polymorphism.</li> <li>4. Allows a class to implement multiple interfaces.</li> <li>5. Improves testability through interchangeable implementations/mocks.</li> <li>6. Helps separate specification from implementation.</li> <li>7. Makes systems easier to extend with new implementations.</li> <li>8. Useful for API and framework design.</li> <li>9. Reduces dependency on concrete classes.</li> </ol>                                                                     |
| <b>Disadvantages</b>       | <ol style="list-style-type: none"> <li>1. Can introduce unnecessary abstraction for very small/simple applications.</li> <li>2. Large interfaces can become difficult to implement and maintain.</li> <li>3. Excessive interfaces may make a codebase harder to navigate.</li> <li>4. An interface generally does not provide the shared instance state that an abstract class can provide.</li> <li>5. Changing a widely used interface can affect many implementations and clients.</li> <li>6. Poorly designed interfaces can violate the Interface Segregation Principle and force classes to implement irrelevant methods.</li> </ol> |
| <b>When to use</b>         | Use interfaces when multiple classes need to follow the <b>same contract</b> , especially when implementations may vary, classes are unrelated, or multiple capabilities need to be combined.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>When not to use</b>     | Avoid creating an interface when there is only one implementation and no meaningful abstraction requirement, or when substantial shared state and implementation are better represented by an abstract/concrete class.                                                                                                                                                                                                                                                                                                                                                                                                                     |

|                                                    |                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Real-time use case 1 — Payment</b>              | Define Payment with pay(). CreditCardPayment, UPIPayment, and NetBankingPayment can implement it differently. The business layer depends only on Payment.                                                                                                                                                                                                                        |
| <b>Real-time use case 2 — Notification</b>         | NotificationSender can be implemented by EmailSender, SmsSender, and PushNotificationSender. The application can switch notification mechanisms without changing the calling code.                                                                                                                                                                                               |
| <b>Real-time use case 3 — Database access</b>      | A UserRepository interface can have implementations such as a database repository, in-memory repository, or test/mock repository.                                                                                                                                                                                                                                                |
| <b>Real-time use case 4 — Logging</b>              | A Logger interface can have console, file, cloud, or test implementations. Application code depends on the logging contract rather than a specific logger.                                                                                                                                                                                                                       |
| <b>Real-time use case 5 — File storage</b>         | StorageService can be implemented by local-file storage, cloud storage, or an object-storage implementation.                                                                                                                                                                                                                                                                     |
| <b>Real-time use case 6 — Hardware abstraction</b> | A Printer interface can provide a common print() operation for different printer models or drivers.                                                                                                                                                                                                                                                                              |
| <b>Real-time use case 7 — Java Collections</b>     | List, Set, and Map are interfaces that allow code to work with different implementations such as ArrayList, LinkedList, HashSet, and HashMap.                                                                                                                                                                                                                                    |
| <b>Java example</b>                                | <pre>interface Payment {     void pay(double amount); }  class UpiPayment implements Payment {     public void pay(double amount) {         System.out.println("Paid ₹" + amount + " using UPI");     } }  class CreditCardPayment implements Payment {     public void pay(double amount) {         System.out.println("Paid ₹" + amount + " using Credit Card");     } }</pre> |
| <b>Polymorphic usage</b>                           | Payment payment = new UpiPayment();payment.pay(1000);The same Payment reference can point to CreditCardPayment, UPIPayment, or another implementation.                                                                                                                                                                                                                           |
| <b>Alternate feature — Abstract Class</b>          | An <b>abstract class</b> is the closest language-level alternative when you need both a contract and shared implementation/state. Unlike                                                                                                                                                                                                                                         |

|                                           |                                                                                                                                                                                                                              |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                           | an interface, a class can extend only one abstract class.                                                                                                                                                                    |
| <b>Alternate feature — Composition</b>    | <b>Composition</b> can replace interface-based or inheritance-based designs when behavior is better modeled by assembling independent objects.                                                                               |
| <b>Alternate feature — Concrete Class</b> | A normal concrete class is appropriate when there is one well-defined implementation and no need for interchangeable implementations or a separate contract.                                                                 |
| <b>Interface vs Abstract Class</b>        | <b>Interface:</b> best for a contract/capability and multiple inheritance of type. <b>Abstract class:</b> best for closely related classes requiring shared state, constructors, and implementation.                         |
| <b>Interface vs Composition</b>           | <b>Interface:</b> defines a replaceable contract. <b>Composition:</b> delegates work to contained objects. In many modern designs, interfaces + composition work particularly well together.                                 |
| <b>Best practice</b>                      | Keep interfaces <b>small, cohesive, and focused on a specific responsibility</b> . Program against interfaces when substitution or abstraction provides real value; avoid creating interfaces merely because a class exists. |

| Aspect                 | Exceptions                                                                                                                                                                                                                                                           |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Java Version</b>    | Java 1.0                                                                                                                                                                                                                                                             |
| <b>Year</b>            | 1996                                                                                                                                                                                                                                                                 |
| <b>Feature</b>         | Exceptions / Exception Handling                                                                                                                                                                                                                                      |
| <b>UniqueClassName</b> | ExceptionHandler                                                                                                                                                                                                                                                     |
| <b>Definition</b>      | <b>Exception handling</b> is Java's mechanism for detecting, propagating, and handling abnormal conditions during program execution without necessarily terminating the entire application. Java uses try, catch, finally, throw, and throws for exception handling. |
| <b>Main purpose</b>    | To separate <b>error-handling logic from normal business logic</b> and provide a controlled way to recover from or report exceptional situations.                                                                                                                    |
| <b>Main keywords</b>   | try, catch, finally, throw, throws                                                                                                                                                                                                                                   |



|                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Main hierarchy</b>                             | Throwable → Exception and Error. RuntimeException is a major subclass of Exception for unchecked exceptions.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Checked exceptions</b>                         | Exceptions other than RuntimeException and its subclasses are generally checked by the compiler. A method must catch them or declare them with throws.                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Unchecked exceptions</b>                       | RuntimeException and its subclasses are unchecked. Examples include NullPointerException, IllegalArgumentException, and ArithmeticException.                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Advantages</b>                                 | <ol style="list-style-type: none"> <li>1. Separates error handling from normal code.</li> <li>2. Prevents many failures from abruptly terminating the application</li> <li>3. Allows errors to propagate to an appropriate handling layer.</li> <li>4. Provides meaningful error information.</li> <li>5. Supports recovery from expected exceptional situations.</li> <li>6. Allows custom exception types for domain-specific errors.</li> <li>7. finally can be used for cleanup operations.</li> <li>8. Improves maintainability when exception boundaries are designed properly.</li> </ol>                      |
| <b>Disadvantages</b>                              | <ol style="list-style-type: none"> <li>1. Excessive exception handling can make code complicated.</li> <li>2. Exceptions have runtime overhead when actually thrown and processed.</li> <li>3. Using exceptions for ordinary control flow can make programs harder to understand.</li> <li>4. Catching overly broad exceptions such as Exception can hide programming errors.</li> <li>5. Poor exception messages make troubleshooting difficult.</li> <li>6. Swallowing exceptions without logging or handling them can hide failures.</li> <li>7. Deeply nested try-catch blocks can reduce readability.</li> </ol> |
| <b>When to use</b>                                | Use exceptions when an operation encounters an <b>abnormal condition</b> that the current code cannot reasonably handle through its normal return value or control flow.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>When not to use</b>                            | Do not use exceptions for normal program flow, routine conditions that are expected and easily represented by normal values, or simply to avoid writing appropriate validation/control logic.                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Real-time use case 1 — File handling</b>       | A file may not exist or may be inaccessible. Java can use IOException/related exceptions to report the failure and allow the application to respond appropriately.                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Real-time use case 2 — Database operations</b> | Database connectivity, SQL, transaction, or constraint failures can be propagated as exceptions so the application can roll back, retry where appropriate, or report the failure.                                                                                                                                                                                                                                                                                                                                                                                                                                     |

|                                                            |                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Real-time use case 3 — Network communication</b>        | Network connections can fail because of timeouts, unavailable servers, or interrupted communication. Exceptions allow these failures to be handled separately from successful communication.                                                                        |
| <b>Real-time use case 4 — Payment processing</b>           | A payment service may reject a transaction. A domain-specific exception such as <code>PaymentFailedException</code> can communicate the failure to the appropriate application layer.                                                                               |
| <b>Real-time use case 5 — Input validation</b>             | An API or service can throw an appropriate exception when supplied data violates required constraints, such as an invalid argument or illegal state.                                                                                                                |
| <b>Real-time use case 6 — Authentication/authorization</b> | Application/service layers can use specific exception types to propagate authentication or authorization failures to a layer responsible for producing the appropriate response.                                                                                    |
| <b>Real-time use case 7 — Resource cleanup</b>             | <code>finally</code> or <code>try-with-resources</code> can ensure resources such as streams and connections are properly closed even when an operation fails.                                                                                                      |
| <b>Simple Java example</b>                                 | <pre>try {     int result = 10 / 0; } catch (ArithmeticException e) {     System.out.println("Cannot divide by zero"); }</pre>                                                                                                                                      |
| <b>Example with throw</b>                                  | <pre>if (amount &lt; 0) {     throw new IllegalArgumentException("Amount cannot be negative"); }</pre>                                                                                                                                                              |
| <b>Example with throws</b>                                 | <pre>void readFile() throws IOException {     // file-reading operation }</pre>                                                                                                                                                                                     |
| <b>Custom exception example</b>                            | <pre>class PaymentFailedException extends Exception {     PaymentFailedException(String message) {         super(message);     } }</pre>                                                                                                                            |
| <b>Exception propagation</b>                               | If a method cannot appropriately handle an exception, it can propagate the exception to its caller using <code>throws</code> . This allows handling at a higher architectural layer.                                                                                |
| <b>Alternate feature — Return values</b>                   | <b>Return values/status objects</b> can represent expected outcomes without exceptions. For example, a method can return a result containing success, <code>errorCode</code> , and <code>message</code> . This is often preferable for routinely expected failures. |
| <b>Alternate feature —</b>                                 | <code>Optional&lt;T&gt;</code> can represent the <b>presence or absence of a value</b>                                                                                                                                                                              |

|                                                  |                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Optional</b>                                  | when absence is an expected outcome. It is not a general replacement for exceptions.                                                                                                                                                                                                 |
| <b>Alternate feature — Result/Either pattern</b> | A <b>Result/Either-style type</b> explicitly represents success or failure as data, making error handling visible in the method's return type. Java does not have a built-in general-purpose Result type, but applications can define one.                                           |
| <b>Alternate feature — Validation</b>            | <b>Validation before performing an operation</b> can prevent predictable errors. For example, validate that an amount is positive before processing a payment.                                                                                                                       |
| <b>Alternate feature — Assertions</b>            | assert can verify assumptions during development/testing, but assertions are <b>not a replacement for production exception handling or user-input validation</b> .                                                                                                                   |
| <b>Exceptions vs Return Values</b>               | <b>Exceptions:</b> suitable for exceptional/unrecoverable-at-current-layer situations. <b>Return values:</b> often better for expected business outcomes that callers routinely need to evaluate.                                                                                    |
| <b>Exceptions vs Optional</b>                    | <b>Exceptions:</b> communicate failures. <b>Optional:</b> communicates that a value may be absent. They solve different problems.                                                                                                                                                    |
| <b>Best practice</b>                             | Catch exceptions at the layer that can meaningfully handle them, use specific exception types, preserve useful context, avoid empty catch blocks, don't use exceptions for ordinary control flow, and don't catch Error unless there is a very specific infrastructure-level reason. |

| Aspect          | Thread Basics                                                                                                                                                                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 1.0                                                                                                                                                                                                                                                          |
| Year            | 1996                                                                                                                                                                                                                                                              |
| Feature         | Thread Basics                                                                                                                                                                                                                                                     |
| UniqueClassName | ThreadBasics                                                                                                                                                                                                                                                      |
| Definition      | <b>Thread basics</b> provide Java's fundamental mechanism for executing multiple sequences of instructions concurrently within a process. Java 1.0 introduced Thread, Runnable, thread lifecycle operations, priorities, synchronization, and related primitives. |
| Main purpose    | To perform multiple tasks concurrently, improve application                                                                                                                                                                                                       |

|                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                               | responsiveness, and make better use of available CPU resources.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Basic creation — extending Thread</b>      | <pre> class Worker extends Thread {     public void run() {         System.out.println("Task running");     } } new Worker().start(); </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Basic creation — implementing Runnable</b> | <pre> class Worker implements Runnable {     public void run() {         System.out.println("Task running");     } } new Thread(new Worker()).start(); </pre>                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Important rule</b>                         | Call <b>start()</b> , not <b>run()</b> , to start a new thread. Calling <b>run()</b> directly executes the method on the current thread.                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Typical lifecycle</b>                      | <b>New → Runnable → Running → Blocked/Waiting/Timed Waiting → Runnable → Terminated.</b> Exact scheduling and state transitions are controlled by the JVM and operating system.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Advantages</b>                             | <ol style="list-style-type: none"> <li>1. Enables concurrent execution.</li> <li>2. Can improve responsiveness of applications.</li> <li>3. Allows independent tasks to make progress concurrently.</li> <li>4. Can utilize multiple CPU cores when tasks are actually executed in parallel.</li> <li>5. Useful for background operations.</li> <li>6. Provides synchronization mechanisms for coordinating shared data.</li> <li>7. Fundamental building block for Java's later concurrency abstractions.</li> </ol>                                                                      |
| <b>Disadvantages</b>                          | <ol style="list-style-type: none"> <li>1. Threads have creation and scheduling overhead.</li> <li>2. Too many threads can consume significant memory and CPU resources.</li> <li>3. Shared mutable data can cause race conditions.</li> <li>4. Incorrect synchronization can cause deadlocks or thread contention.</li> <li>5. Concurrent programs are harder to debug and test.</li> <li>6. Thread scheduling is not fully predictable.</li> <li>7. Low-level thread management can produce complex code.</li> <li>8. Blocking operations can waste platform-thread resources.</li> </ol> |
| <b>When to use</b>                            | Use basic threads when you need relatively direct control over concurrent execution, especially for simple background or independent tasks and when higher-level concurrency APIs are unnecessary or unavailable.                                                                                                                                                                                                                                                                                                                                                                          |

|                                                        |                                                                                                                                                                                                                                                                                                  |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>When not to use</b>                                 | Avoid manually creating large numbers of threads for application workloads. Avoid low-level threads when thread pools, executors, asynchronous APIs, or other higher-level concurrency abstractions provide safer and easier management.                                                         |
| <b>Real-time use case 1 — Background processing</b>    | A desktop application can perform a long-running calculation on a background thread so that the user interface remains responsive.                                                                                                                                                               |
| <b>Real-time use case 2 — File processing</b>          | A program can process independent files concurrently, provided shared resources are properly coordinated.                                                                                                                                                                                        |
| <b>Real-time use case 3 — Network operations</b>       | A server can use separate threads to handle independent client connections or network tasks.                                                                                                                                                                                                     |
| <b>Real-time use case 4 — Logging</b>                  | A background thread can process queued log messages while the main application continues performing its primary work.                                                                                                                                                                            |
| <b>Real-time use case 5 — Data processing</b>          | Independent computational tasks can execute concurrently, potentially improving throughput on multi-core systems.                                                                                                                                                                                |
| <b>Real-time use case 6 — Periodic/background task</b> | A background thread can perform periodic work such as checking application state or refreshing data. For modern applications, scheduled executors are generally preferable to manually managing such threads.                                                                                    |
| <b>Simple Java example</b>                             | <pre> class Worker extends Thread {     @Override     public void run() {         System.out.println("Worker thread is running");     } }  public class ThreadBasics {     public static void main(String[] args) {         Worker worker = new Worker();         worker.start();     } } </pre> |
| <b>What happens</b>                                    | main() creates a Worker thread and calls start(). The JVM schedules the new thread, which eventually executes its run() method independently of the main thread.                                                                                                                                 |
| <b>Synchronization</b>                                 | When multiple threads access shared mutable state, synchronization mechanisms such as synchronized can be used to coordinate access and establish appropriate memory-visibility guarantees.                                                                                                      |
| <b>Example of</b>                                      | <pre> synchronized void increment() {     count++; } </pre>                                                                                                                                                                                                                                      |

|                                              |                                                                                                                                                                                                                                                                                               |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>synchronization</b>                       | <pre>}</pre> <p>This can prevent multiple threads from simultaneously executing the critical section on the same object.</p>                                                                                                                                                                  |
| <b>Real-time caution</b>                     | Creating one platform thread for every task does not automatically improve performance. For many concurrent tasks, a bounded thread pool or another higher-level concurrency mechanism is usually more appropriate.                                                                           |
| <b>Alternate feature — ExecutorService</b>   | <b>ExecutorService</b> is a higher-level concurrency abstraction for submitting tasks to managed thread pools. It is generally preferable to manually creating a new thread for every task.                                                                                                   |
| <b>Alternate feature — ForkJoinPool</b>      | <b>ForkJoinPool</b> is useful for parallel, recursively decomposable computational tasks and supports work-stealing.                                                                                                                                                                          |
| <b>Alternate feature — CompletableFuture</b> | <b>CompletableFuture</b> provides composable asynchronous workflows and can avoid manually coordinating many individual threads.                                                                                                                                                              |
| <b>Alternate feature — Virtual Threads</b>   | <b>Virtual threads</b> , introduced much later in Java 21, provide lightweight threads particularly suited to applications with large numbers of mostly blocking I/O tasks. They are a modern alternative to managing large numbers of platform threads directly.                             |
| <b>Alternate feature — Parallel Streams</b>  | <b>Parallel streams</b> can express certain data-parallel operations concisely, although they should be used carefully because they use shared/common execution infrastructure and are not suitable for every workload.                                                                       |
| <b>Thread vs ExecutorService</b>             | <p><b>Thread:</b> low-level and gives direct lifecycle control.</p> <p><b>ExecutorService:</b> higher-level task execution and thread-pool management; usually preferable for application workloads.</p>                                                                                      |
| <b>Thread vs Virtual Thread</b>              | <p><b>Platform thread:</b> relatively heavyweight and backed by an OS thread. <b>Virtual thread:</b> lightweight JVM-managed thread designed to support very high concurrency, especially for blocking I/O.</p>                                                                               |
| <b>Best practice</b>                         | Prefer <b>task-based concurrency</b> (ExecutorService, futures, structured concurrency where appropriate, etc.) over manually creating threads for every task. Keep shared mutable state minimal and use appropriate synchronization or concurrent data structures when sharing is necessary. |

| Aspect                     | Synchronization                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version               | Java 1.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Year                       | 1996                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Feature                    | Synchronization                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| UniqueClassName            | SynchronizationBasics                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Definition                 | <b>Synchronization</b> is Java's mechanism for controlling concurrent access to shared mutable data so that multiple threads do not simultaneously execute conflicting operations on the same critical section. It also provides important <b>memory-visibility and ordering guarantees</b> between synchronized operations.                                                                                                                                                                      |
| Main purpose               | To prevent <b>race conditions</b> , maintain data consistency, and coordinate access to shared resources among threads.                                                                                                                                                                                                                                                                                                                                                                           |
| Primary keyword            | synchronized                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Forms                      | Java supports synchronized <b>instance methods, static methods, and synchronized blocks</b> .                                                                                                                                                                                                                                                                                                                                                                                                     |
| Instance method example    | synchronized void increment() { count++; }                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Synchronized block example | synchronized (lock) { count++; }                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Static synchronization     | static synchronized void update() { ... } synchronizes using the Class object's monitor rather than an individual instance.                                                                                                                                                                                                                                                                                                                                                                       |
| Advantages                 | <ol style="list-style-type: none"> <li>1. Prevents many race conditions.</li> <li>2. Protects critical sections involving shared mutable state.</li> <li>3. Provides memory visibility between synchronized operations using the same monitor.</li> <li>4. Helps maintain data consistency.</li> <li>5. Built directly into the Java language.</li> <li>6. Can be applied selectively with synchronized blocks.</li> <li>7. Makes certain thread-safety requirements explicit in code.</li> </ol> |
| Disadvantages              | <ol style="list-style-type: none"> <li>1. Threads may have to wait for a lock, reducing concurrency.</li> <li>2. Excessive synchronization can reduce application performance.</li> <li>3. Incorrect lock design can cause <b>deadlocks</b>.</li> <li>4. Lock contention can become significant under heavy concurrency.</li> <li>5. Large synchronized sections can unnecessarily serialize work.</li> <li>6. Nested locks can make concurrency logic difficult to reason</li> </ol>             |

|                                                              |                                                                                                                                                                                                      |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                              | <p>about.</p> <p>7. Synchronization does not automatically make an entire class thread-safe; all relevant shared-state access must be designed consistently.</p>                                     |
| <b>When to use</b>                                           | Use synchronization when multiple threads access <b>shared mutable state</b> and the operations require mutual exclusion or a happens-before relationship.                                           |
| <b>When not to use</b>                                       | Avoid synchronization when data is immutable, thread-confined, or safely accessed through another concurrency mechanism. Do not synchronize unnecessarily around independent operations.             |
| <b>Real-time use case 1 — Bank account</b>                   | Multiple threads may attempt to withdraw money from the same account. Synchronization can ensure that balance checking and balance modification happen atomically with respect to other withdrawals. |
| <b>Real-time use case 2 — Ticket booking</b>                 | Two customers may attempt to book the same seat simultaneously. A synchronized critical section can ensure that only one booking operation changes the seat's availability at a time.                |
| <b>Real-time use case 3 — Inventory management</b>           | Concurrent orders may decrease the same product's stock. Synchronization can protect the stock update so inventory does not become inconsistent.                                                     |
| <b>Real-time use case 4 — Shared counter</b>                 | Multiple worker threads increment a shared counter. Synchronization prevents lost updates when the increment operation is not otherwise atomic.                                                      |
| <b>Real-time use case 5 — Shared cache</b>                   | Multiple threads may update shared cache state. Synchronization can protect compound operations such as checking, modifying, and publishing an entry.                                                |
| <b>Real-time use case 6 — Singleton-style initialization</b> | Synchronization can be involved in safely coordinating initialization, although modern Java provides simpler patterns such as initialization-on-demand holders and enum-based singletons.            |
| <b>Simple Java example</b>                                   | <pre> class Counter {     private int count;      synchronized void increment() {         count++;     }      int getCount() {         return count;     } } </pre>                                  |



|                                                   |                                                                                                                                                                                                                      |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                   | <pre>         }     } </pre>                                                                                                                                                                                         |
| <b>How the example works</b>                      | If multiple threads call increment() on the <b>same Counter object</b> , only one thread at a time can execute the synchronized method. This prevents concurrent updates from interfering with each other.           |
| <b>Synchronized block example</b>                 | <pre> synchronized (lock) {     balance -= amount; } </pre>                                                                                                                                                          |
| <b>Why use a block?</b>                           | A synchronized block allows you to protect only the <b>critical section</b> instead of locking an entire method, potentially reducing unnecessary lock contention.                                                   |
| <b>Important limitation</b>                       | Synchronizing different blocks on different lock objects does not coordinate them. Threads that need to protect the same shared state must use a compatible locking strategy.                                        |
| <b>Alternate feature — Lock</b>                   | java.util.concurrent.locks.Lock and implementations such as ReentrantLock provide explicit lock management and additional capabilities such as tryLock(), interruptible acquisition, and optional fairness policies. |
| <b>Alternate feature — Atomic classes</b>         | Classes such as AtomicInteger, AtomicLong, and AtomicReference provide lock-free/low-level atomic operations for suitable individual shared variables. Example: counter.incrementAndGet();                           |
| <b>Alternate feature — Concurrent collections</b> | Classes such as ConcurrentHashMap, ConcurrentLinkedQueue, and blocking queues provide thread-safe data structures without requiring application code to synchronize every access manually.                           |
| <b>Alternate feature — Immutable objects</b>      | <b>Immutability</b> can eliminate the need for synchronization because an object's state cannot change after construction.                                                                                           |
| <b>Alternate feature — Thread confinement</b>     | Keeping mutable data confined to a single thread avoids concurrent access and therefore can eliminate the need for synchronization around that data.                                                                 |
| <b>Alternate feature — volatile</b>               | volatile can provide visibility and ordering for a variable between threads, but it <b>does not provide general mutual exclusion</b> or make compound operations such as count++ atomic.                             |
| <b>Synchronization vs Lock</b>                    | synchronized is simpler and automatically releases the monitor when leaving the synchronized region. Lock provides more advanced control but requires careful explicit unlocking, normally using try/finally.        |

|                                          |                                                                                                                                                                                                                                                                                                        |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Synchronization vs Atomic classes</b> | Use atomic classes for suitable individual atomic state transitions; use synchronization or locks when you need to protect a <b>compound invariant involving multiple operations or pieces of state</b> .                                                                                              |
| <b>Synchronization vs Immutability</b>   | Synchronization coordinates mutable shared state; immutability avoids many concurrency problems by eliminating state changes altogether.                                                                                                                                                               |
| <b>Best practice</b>                     | Keep critical sections <b>small</b> , use a well-defined private lock where appropriate, avoid nested locks when possible, protect complete invariants rather than individual statements, and prefer higher-level java.util.concurrent utilities when they express the required behavior more clearly. |

| Aspect                | Wait/notify                                                                                                                                                                                                                                                             |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version          | Java 1.0                                                                                                                                                                                                                                                                |
| Year                  | 1996                                                                                                                                                                                                                                                                    |
| Feature               | wait() / notify() / notifyAll()                                                                                                                                                                                                                                         |
| UniqueClassName       | WaitNotify                                                                                                                                                                                                                                                              |
| Definition            | <b>Wait/notify</b> is Java's low-level thread-coordination mechanism in which one thread waits for a condition to become true, while another thread signals that the condition may have changed. These methods are defined by Object and work with an object's monitor. |
| Main purpose          | To coordinate threads that share a resource or condition, particularly in <b>producer-consumer</b> style scenarios.                                                                                                                                                     |
| Methods               | wait() — releases the object's monitor and suspends the current thread until notified/interrupted.notify() — wakes one waiting thread, if any.notifyAll() — wakes all threads waiting on that object's monitor.                                                         |
| Important requirement | A thread must own the object's monitor before calling wait(), notify(), or notifyAll(), normally by executing inside a synchronized method/block. Otherwise, Java throws IllegalMonitorStateException.                                                                  |
| Important rule        | Always call wait() inside a <b>while loop</b> that checks the required                                                                                                                                                                                                  |

|                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                 | condition, rather than assuming that notification means the condition is now true.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Advantages</b>                               | <ol style="list-style-type: none"> <li>1. Built directly into Java's Object class.</li> <li>2. Provides a mechanism for threads to coordinate around shared conditions.</li> <li>3. wait() releases the monitor while waiting, allowing another thread to make progress.</li> <li>4. Useful for implementing low-level producer-consumer algorithms.</li> <li>5. notifyAll() can wake all threads waiting for a condition change.</li> <li>6. Does not require busy-waiting while a thread is waiting.</li> </ol>                                                                                                                                                                                                                                                           |
| <b>Disadvantages</b>                            | <ol style="list-style-type: none"> <li>1. It is a relatively <b>low-level and error-prone</b> concurrency mechanism.</li> <li>2. Requires correct use of synchronization and monitor ownership.</li> <li>3. Incorrect condition handling can cause missed progress or indefinite waiting.</li> <li>4. Calling notify() can wake a thread that cannot make useful progress, depending on the design.</li> <li>5. wait() can return because of <b>spurious wakeups</b>, so a condition must be checked in a loop.</li> <li>6. Complex wait/notify protocols are difficult to understand and maintain.</li> <li>7. Interrupt handling must be designed correctly.</li> <li>8. Higher-level concurrency utilities are usually easier and safer for common use cases.</li> </ol> |
| <b>When to use</b>                              | Use when implementing or maintaining <b>low-level monitor-based coordination</b> , especially legacy Java code or specialized synchronization protocols where higher-level utilities do not fit.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>When not to use</b>                          | Avoid it for ordinary producer-consumer queues, task scheduling, latches, semaphores, or other common coordination problems when java.util.concurrent provides a clearer abstraction.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Real-time use case 1 — Producer/Consumer</b> | A producer waits when a bounded buffer is full, while a consumer waits when the buffer is empty. After changing the buffer state, a thread signals waiting threads.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Real-time use case 2 — Resource pool</b>     | A worker can wait when no database connection/resource is available. When another worker releases a resource, it can signal a waiting thread. Modern applications generally use higher-level pools/queues for this.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Real-time use case 3 — Job processing</b>    | A worker can wait for work to appear in a shared queue. A producer adds a job and signals the worker. A BlockingQueue is normally preferable for this pattern.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

|                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Real-time use case 4 — Shared state transition</b> | One thread may wait until another thread changes a shared object's state from NOT_READY to READY, then signals waiting threads.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Real-time use case 5 — Legacy systems</b>          | Older Java applications may use monitor-based wait/notify for coordination. Understanding it is important when maintaining such systems.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Simple Java example</b>                            | <pre> class Message {     private String message;     private boolean available = false;      synchronized void put(String message) {         while (available) {             try { wait(); } catch (InterruptedException e) {                 Thread.currentThread().interrupt();             }             return;         }         this.message = message;         available = true;         notifyAll();     }      synchronized String get() throws     InterruptedException {         while (!available) {             wait();         }         available = false;         notifyAll();         return message;     } } </pre> |
| <b>How the example works</b>                          | get() waits while no message is available. put() stores a message, changes available to true, and calls notifyAll(). The waiting consumer wakes, reacquires the monitor, rechecks the condition, and retrieves the message.                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Why while instead of if?</b>                       | A waiting thread may wake without the desired condition being true, including because of a spurious wakeup. After reacquiring the monitor, it must <b>recheck the condition</b> .                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>notify() vs notifyAll()</b>                        | notify() wakes <b>one</b> waiting thread. notifyAll() wakes <b>all</b> threads waiting on that monitor. notifyAll() is often safer when multiple different conditions may be associated with the same monitor, although the awakened threads still need to recheck their                                                                                                                                                                                                                                                                                                                                                               |

|                                                   |                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                   | conditions.                                                                                                                                                                                                                                                                                              |
| <b>Important distinction</b>                      | notify()/notifyAll() do <b>not release the lock immediately</b> . The notifying thread continues until it exits the synchronized region; a woken thread must then reacquire the monitor before continuing.                                                                                               |
| <b>Alternate feature — BlockingQueue</b>          | <b>BlockingQueue</b> is usually the preferred alternative for producer-consumer communication. Methods such as put() and take() automatically coordinate waiting when the queue is full or empty.                                                                                                        |
| <b>Alternate feature — Lock + Condition</b>       | ReentrantLock with Condition provides an explicit alternative to intrinsic monitors and supports separate condition queues, such as notEmpty and notFull.                                                                                                                                                |
| <b>Alternate feature — CountdownLatch</b>         | <b>CountDownLatch</b> is appropriate when one or more threads need to wait until a fixed number of operations/events complete.                                                                                                                                                                           |
| <b>Alternate feature — Semaphore</b>              | <b>Semaphore</b> is appropriate when threads need controlled access to a limited number of permits/resources.                                                                                                                                                                                            |
| <b>Alternate feature — CyclicBarrier / Phaser</b> | These are useful when multiple threads need to rendezvous at one or more synchronization points.                                                                                                                                                                                                         |
| <b>Alternate feature — CompletableFuture</b>      | For asynchronous workflows, CompletableFuture can express dependencies between tasks without manually managing wait/notify protocols.                                                                                                                                                                    |
| <b>Wait/notify vs BlockingQueue</b>               | <b>Wait/notify</b> : low-level monitor coordination. <b>BlockingQueue</b> : high-level producer-consumer abstraction and generally easier to use correctly.                                                                                                                                              |
| <b>Wait/notify vs Lock/Condition</b>              | <b>Wait/notify</b> : tied to an object's intrinsic monitor. <b>Condition</b> : provides explicit condition objects associated with a Lock, allowing more structured coordination.                                                                                                                        |
| <b>Wait/notify vs CountdownLatch</b>              | <b>Wait/notify</b> : general-purpose condition coordination. <b>CountDownLatch</b> : specifically represents a one-time countdown completion condition.                                                                                                                                                  |
| <b>Best practice</b>                              | Prefer <b>higher-level concurrency utilities</b> whenever they naturally express the problem. If using wait()/notify()/notifyAll(), protect the condition with the appropriate monitor, test conditions in while loops, handle interruption correctly, and carefully document the shared-state protocol. |

| Aspect                 | Arrays                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version           | Java 1.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Year                   | 1996                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Feature                | Arrays                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| UniqueClassName        | Arrays                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Definition             | An <b>array</b> is a fixed-size, indexed collection that stores multiple values of the <b>same type</b> . The elements are accessed using an integer index starting from 0. Java arrays can store primitives or object references.                                                                                                                                                                                                                                                                                                                                             |
| Main purpose           | To store and efficiently access a known number of related values under a single variable name.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Basic syntax           | <code>int[] numbers = new int[5];</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Initialization example | <code>int[] numbers = {10, 20, 30, 40, 50};</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Access example         | <code>System.out.println(numbers[0]); // 10</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Array size             | Fixed after creation. <code>numbers.length</code> returns the number of elements.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Indexing               | Zero-based: the first element is at index 0, and the last element is at <code>length - 1</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| Advantages             | <ol style="list-style-type: none"> <li>1. Simple and easy to use.</li> <li>2. Fast indexed access, typically <b>O(1)</b>.</li> <li>3. Low structural overhead compared with many collection types.</li> <li>4. Can store primitive values directly, avoiding boxing in primitive arrays.</li> <li>5. Predictable memory layout and performance characteristics.</li> <li>6. Useful for fixed-size data.</li> <li>7. Multidimensional arrays are supported through arrays of arrays.</li> <li>8. Widely supported by Java APIs and language features.</li> </ol>                |
| Disadvantages          | <ol style="list-style-type: none"> <li>1. Size is fixed after creation.</li> <li>2. Inserting or removing elements requires shifting/copying when maintaining a logical sequence.</li> <li>3. Provides fewer high-level operations than collection classes.</li> <li>4. All elements must have the same array component type.</li> <li>5. Incorrect indexing can cause <code>ArrayIndexOutOfBoundsException</code>.</li> <li>6. Resizing requires creating another array and copying elements.</li> <li>7. Managing dynamic collections manually can make code more</li> </ol> |

|                                                           |                                                                                                                                                                                                                               |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                           | complicated.                                                                                                                                                                                                                  |
| <b>When to use</b>                                        | Use arrays when the <b>number of elements is known/fixed</b> , when direct indexed access is important, when working with primitive data efficiently, or when an API specifically requires an array.                          |
| <b>When not to use</b>                                    | Avoid arrays when the collection needs frequent insertion/removal, dynamic resizing, rich collection operations, or when the number of elements is highly unpredictable.                                                      |
| <b>Real-time use case 1 — Student marks</b>               | <code>int[] marks = {85, 92, 78, 88, 95};</code> can represent a fixed set of marks for a known group of subjects.                                                                                                            |
| <b>Real-time use case 2 — Monthly data</b>                | <code>double[] sales = new double[12];</code> can represent sales for the 12 months of a year.                                                                                                                                |
| <b>Real-time use case 3 — Image pixels</b>                | Image-processing software can use arrays to represent pixel data, such as <code>int[]</code> or multidimensional/array-based structures for image channels.                                                                   |
| <b>Real-time use case 4 — Matrix calculations</b>         | <code>int[][] matrix = new int[3][3];</code> can represent a fixed-size matrix used in mathematical or scientific calculations.                                                                                               |
| <b>Real-time use case 5 — Sensor readings</b>             | A fixed-size <code>double[]</code> can hold a known number of readings collected during a sampling interval.                                                                                                                  |
| <b>Real-time use case 6 — Lookup tables</b>               | Arrays can efficiently store values that are accessed using an integer index, such as precomputed configuration or lookup data.                                                                                               |
| <b>Real-time use case 7 — Character/string processing</b> | <code>char[]</code> can represent a mutable sequence of characters when an API or algorithm specifically benefits from character-array access.                                                                                |
| <b>Simple Java example</b>                                | <pre> public class Arrays {     public static void main(String[] args) {         int[] numbers = {10, 20, 30, 40, 50};          for (int number : numbers) {             System.out.println(number);         }     } } </pre> |
| <b>What happens</b>                                       | The array stores five <code>int</code> values. The enhanced for loop reads each element sequentially without manually managing the index.                                                                                     |
| <b>Multidimensional array example</b>                     | <pre> int[][] matrix = {     {1, 2, 3},     {4, 5, 6} }; </pre>                                                                                                                                                               |

|                                       |                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                       | <code>System.out.println(matrix[1][2]); // 6</code>                                                                                                                                                                                                                                                                        |
| <b>Array of objects</b>               | <code>Employee[] employees = new Employee[3];</code> stores references to <code>Employee</code> objects. The array itself has a fixed length, while the referenced objects can have mutable state.                                                                                                                         |
| <b>Important characteristic</b>       | Arrays are <b>covariant for reference types</b> . For example, <code>String[]</code> can be assigned to an <code>Object[]</code> reference. However, storing an incompatible object through that reference can produce <code>ArrayStoreException</code> .                                                                  |
| <b>Alternate feature — ArrayList</b>  | <b>ArrayList</b> is the most common alternative when you need a dynamically resizable list with convenient methods such as <code>add()</code> , <code>remove()</code> , <code>contains()</code> , and <code>get()</code> .                                                                                                 |
| <b>Alternate feature — LinkedList</b> | <b>LinkedList</b> can be useful for specific workloads involving frequent insertion/removal through known list positions or ends, although it is often less efficient than <code>ArrayList</code> for random access.                                                                                                       |
| <b>Alternate feature — HashSet</b>    | Use <b>HashSet</b> when uniqueness matters rather than indexed access.                                                                                                                                                                                                                                                     |
| <b>Alternate feature — HashMap</b>    | Use <b>HashMap</b> when data should be accessed by a key rather than an integer index.                                                                                                                                                                                                                                     |
| <b>Alternate feature — Deque</b>      | Use <b>Deque</b> when the primary requirement is efficient insertion/removal at both ends, such as stack or queue behavior.                                                                                                                                                                                                |
| <b>Array vs ArrayList</b>             | <b>Array</b> : fixed size, supports primitives directly, minimal structure, excellent indexed access. <b>ArrayList</b> : dynamically resizable, richer API, stores objects and therefore uses wrapper types for primitive values.                                                                                          |
| <b>Array vs HashSet</b>               | <b>Array</b> : ordered indexed elements and duplicates allowed. <b>HashSet</b> : designed primarily for uniqueness and set operations, without index-based access.                                                                                                                                                         |
| <b>Array vs HashMap</b>               | <b>Array</b> : integer-indexed access. <b>HashMap</b> : key-value access.                                                                                                                                                                                                                                                  |
| <b>Performance</b>                    | Direct element access is generally <b>O(1)</b> . Searching an unsorted array is generally <b>O(n)</b> ; inserting/removing while preserving order is generally <b>O(n)</b> .                                                                                                                                               |
| <b>Best practice</b>                  | Prefer arrays for <b>fixed-size, index-oriented, performance-sensitive data</b> or APIs that require arrays. For general-purpose dynamic collections, prefer the appropriate <b>Java Collections Framework</b> type such as <code>ArrayList</code> , <code>HashSet</code> , <code>HashMap</code> , or <code>Deque</code> . |



| Aspect          | Strings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 1.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| Year            | 1996                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Feature         | Strings                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| UniqueClassName | StringBasics                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| Definition      | A <b>String</b> in Java is an object of the java.lang.String class that represents a sequence of characters. Strings are <b>immutable</b> , meaning that once a String object is created, its character contents cannot be changed.                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Main purpose    | To represent and manipulate textual data such as names, messages, URLs, identifiers, file paths, JSON/XML text, and user input.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Basic syntax    | String name = "Anurag";                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Object creation | String s1 = "Hello";String s2 = new String("Hello");                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Immutability    | Operations such as concatenation or replacement do not modify the original String; they create or return another String value.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| String Pool     | Java maintains a <b>String Pool</b> for string literals. Identical literals can share the same pooled object, reducing unnecessary object creation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| Advantages      | <ol style="list-style-type: none"> <li>1. Immutable and therefore inherently safe to share between threads.</li> <li>2. Easy and convenient syntax for text processing.</li> <li>3. Rich API for searching, splitting, replacing, comparing, and transforming text.</li> <li>4. String literals are supported directly by the Java language.</li> <li>5. String pooling can reduce memory usage for repeated literals.</li> <li>6. Strings work naturally with concatenation and formatting APIs.</li> <li>7. Immutability makes strings useful as keys in hash-based collections.</li> <li>8. Widely supported throughout the Java platform and libraries.</li> </ol> |
| Disadvantages   | <ol style="list-style-type: none"> <li>1. Immutability means repeated modifications create additional String objects.</li> <li>2. Repeated concatenation in loops can be inefficient if it creates</li> </ol>                                                                                                                                                                                                                                                                                                                                                                                                                                                          |

|                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                   | <p>many intermediate strings.</p> <p>3. Strings can consume substantial memory when very large amounts of text are retained.</p> <p>4. Character-by-character mutation is inconvenient.</p> <p>5. Incorrect use of == for content comparison can produce unexpected results; use equals() for content equality.</p> <p>6. Sensitive data stored as ordinary strings cannot be explicitly cleared by modifying the string contents.</p> |
| <b>When to use</b>                                | Use String for normal textual data that is <b>mostly read or replaced as whole values</b> , such as names, labels, messages, identifiers, commands, and formatted output.                                                                                                                                                                                                                                                              |
| <b>When not to use</b>                            | Avoid repeatedly modifying a string inside a loop or performing extensive incremental text construction with +. Use StringBuilder or another appropriate mechanism instead.                                                                                                                                                                                                                                                            |
| <b>Real-time use case 1 — User information</b>    | String customerName = "Rahul"; can store names, addresses, email addresses, and other textual information.                                                                                                                                                                                                                                                                                                                             |
| <b>Real-time use case 2 — API requests</b>        | URLs, HTTP headers, query parameters, JSON fragments, and API identifiers commonly involve strings.                                                                                                                                                                                                                                                                                                                                    |
| <b>Real-time use case 3 — Logging</b>             | Applications construct textual log messages using strings: String message = "Order processed: " + orderId;                                                                                                                                                                                                                                                                                                                             |
| <b>Real-time use case 4 — File paths</b>          | File names and paths can be represented using strings, although Path is preferable for filesystem operations requiring path-specific behavior.                                                                                                                                                                                                                                                                                         |
| <b>Real-time use case 5 — Database operations</b> | SQL statements and textual database values can be represented as strings. Parameterized queries should be used rather than constructing SQL by unsafe string concatenation.                                                                                                                                                                                                                                                            |
| <b>Real-time use case 6 — Configuration</b>       | Configuration values such as environment names, feature flags, host names, and property values are commonly represented as strings.                                                                                                                                                                                                                                                                                                    |
| <b>Real-time use case 7 — Validation</b>          | Strings are used to validate formats such as usernames, product codes, identifiers, and other textual input.                                                                                                                                                                                                                                                                                                                           |
| <b>Simple Java example</b>                        | <pre> public class StringBasics {     public static void main(String[] args) {         String name = "Java";         String message = "Hello, " + name;          System.out.println(message);         System.out.println(name.length());         System.out.println(name.toUpperCase());     } } </pre>                                                                                                                                |

|                                          |                                                                                                                                                                                                                                                             |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                          | <pre>         }     } </pre>                                                                                                                                                                                                                                |
| <b>String comparison</b>                 | Use equals() for content comparison: if (name.equals("Java")) { ... }. Avoid using == when the intention is to compare string contents.                                                                                                                     |
| <b>String concatenation</b>              | String result = "Java" + " Programming"; creates the resulting string value. For many incremental concatenations, StringBuilder is generally more appropriate.                                                                                              |
| <b>StringBuilder</b>                     | StringBuilder is <b>mutable</b> and is designed for efficient incremental character-sequence construction, especially in loops. Example: <pre> StringBuilder sb = new StringBuilder(); sb.append("Java"); sb.append(" 1.0"); </pre>                         |
| <b>StringBuffer</b>                      | StringBuffer is a mutable character sequence whose methods are synchronized. It can be useful when a single mutable buffer must be shared with synchronization, although StringBuilder is generally preferred when external synchronization is unnecessary. |
| <b>Char array</b>                        | char[] can be used when direct mutable character storage is required. Example: char[] chars = {'J', 'a', 'v', 'a'};.                                                                                                                                        |
| <b>CharSequence</b>                      | CharSequence is an interface representing a readable sequence of characters. It allows APIs to accept String, StringBuilder, StringBuffer, and other character-sequence implementations.                                                                    |
| <b>Alternate feature — StringBuilder</b> | <b>Primary alternative for frequently changing text.</b> It avoids creating a new immutable string for every append operation and is generally the preferred choice for single-threaded string construction.                                                |
| <b>Alternate feature — StringBuffer</b>  | <b>Thread-aware mutable alternative</b> to StringBuilder, with synchronized methods. Use only when its synchronization characteristics are actually needed.                                                                                                 |
| <b>Alternate feature — char[]</b>        | Useful when characters need to be modified individually or when an API specifically requires a character array.                                                                                                                                             |
| <b>Alternate feature — CharSequence</b>  | Useful when designing APIs that should accept multiple kinds of character sequences rather than requiring specifically a String.                                                                                                                            |
| <b>String vs StringBuilder</b>           | <b>String:</b> immutable, best for stable text values. <b>StringBuilder:</b> mutable, best for repeated text construction.                                                                                                                                  |
| <b>String vs StringBuffer</b>            | <b>String:</b> immutable. <b>StringBuffer:</b> mutable and synchronized. StringBuffer is generally less attractive than StringBuilder for ordinary single-threaded construction.                                                                            |

|                                  |                                                                                                                                                                                                                                                                                                         |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                  |                                                                                                                                                                                                                                                                                                         |
| <b>String vs char[]</b>          | <b>String:</b> immutable, convenient text abstraction. <b>char[]:</b> mutable array of characters with direct indexed access.                                                                                                                                                                           |
| <b>Performance consideration</b> | Individual string operations are usually straightforward, but repeatedly creating strings can increase allocation and garbage-collection pressure. For large or repeated concatenation, prefer StringBuilder or appropriate formatting APIs.                                                            |
| <b>Security consideration</b>    | Do not rely on String for secrets that must be explicitly erased from memory. For certain sensitive credentials, APIs that use mutable character storage may be preferable, although modern credential-handling APIs should be considered where available.                                              |
| <b>Best practice</b>             | Use <b>String as the default type for ordinary immutable text</b> , StringBuilder for repeated/incremental construction, StringBuffer only when its synchronization is required, and specialized types such as Path, URI, or domain-specific value types when the data has semantics beyond plain text. |

| Aspect          | I/O Streams                                                                                                                                                                                                                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version    | Java 1.0                                                                                                                                                                                                                                                                                                                  |
| Year            | 1996                                                                                                                                                                                                                                                                                                                      |
| Feature         | I/O Streams                                                                                                                                                                                                                                                                                                               |
| UniqueClassName | IOStreams                                                                                                                                                                                                                                                                                                                 |
| Definition      | <b>I/O (Input/Output) streams</b> provide Java's original stream-based mechanism for reading data from input sources and writing data to output destinations. Java 1.0 introduced the core java.io stream hierarchy, including byte streams such as InputStream/OutputStream and character streams such as Reader/Writer. |
| Main purpose    | To perform sequential data transfer between an application and sources/destinations such as files, memory, network connections, and other streams.                                                                                                                                                                        |
| Main categories | <b>Byte streams:</b> InputStream, OutputStream — suitable for binary data. <b>Character streams:</b> Reader, Writer — designed for character/text data.                                                                                                                                                                   |

|                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Basic input example</b>                 | <pre>InputStream in = new FileInputStream("input.txt");int data = in.read();</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Basic output example</b>                | <pre>OutputStream out = new<br/>FileOutputStream("output.txt");out.write(65);</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Character input example</b>             | <pre>Reader reader = new FileReader("input.txt");int ch = reader.read();</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Character output example</b>            | <pre>Writer writer = new FileWriter("output.txt");writer.write("Hello Java");</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Advantages</b>                          | <ol style="list-style-type: none"> <li>1. Provides a standard abstraction for sequential I/O.</li> <li>2. Supports both binary and character data.</li> <li>3. Can work with files, memory, network streams, and other sources/destinations.</li> <li>4. Supports <b>stream chaining/decorator-style composition</b> through classes such as <code>BufferedInputStream</code>, <code>BufferedReader</code>, and <code>DataInputStream</code>.</li> <li>5. Buffering can improve I/O efficiency.</li> <li>6. Supports large data without requiring the entire data source to be loaded into memory.</li> <li>7. Provides extensive APIs for file and stream processing.</li> <li>8. Forms the foundation of many older and existing Java I/O APIs.</li> </ol> |
| <b>Disadvantages</b>                       | <ol style="list-style-type: none"> <li>1. The classic java.io APIs can be verbose.</li> <li>2. Incorrect resource management can cause resource leaks.</li> <li>3. Blocking I/O can tie up a platform thread while waiting for data.</li> <li>4. Byte/character encoding must be handled correctly when converting between binary data and text.</li> <li>5. Low-level stream code can require significant error handling.</li> <li>6. Some older classes provide less convenient APIs than newer Java alternatives.</li> <li>7. Sequential stream operations are not ideal for every type of I/O workload.</li> </ol>                                                                                                                                       |
| <b>When to use</b>                         | Use I/O streams when you need <b>sequential reading/writing of bytes or characters</b> , especially for file processing, binary data, serialization-related APIs, or compatibility with existing java.io code.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>When not to use</b>                     | Avoid raw streams when a higher-level API better expresses the operation—for example, use <code>Path/Files</code> for modern filesystem operations, <code>RandomAccessFile</code> or file channels for random access, or NIO/NIO.2 APIs when asynchronous/non-blocking or advanced filesystem features are required.                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Real-time use case 1 — File reading</b> | Read a text file using <code>BufferedReader/Reader</code> or binary data using <code>InputStream</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

|                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Real-time use case 2 — File writing</b>          | Write reports, logs, configuration files, or generated text using <code>Writer</code> or binary data using <code>OutputStream</code> .                                                                                                                                                                                                                                                                                                                                                        |
| <b>Real-time use case 3 — Image processing</b>      | Images, PDFs, ZIP files, and other binary content can be transferred using byte streams.                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Real-time use case 4 — Network communication</b> | Network sockets expose input and output streams for reading incoming bytes and writing outgoing bytes.                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Real-time use case 5 — Data export/import</b>    | Applications can stream CSV, text, binary, or other formats between files and application processes.                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Real-time use case 6 — In-memory processing</b>  | <code>ByteArrayInputStream</code> and <code>ByteArrayOutputStream</code> allow applications to treat an in-memory byte array as a stream.                                                                                                                                                                                                                                                                                                                                                     |
| <b>Real-time use case 7 — Buffered processing</b>   | <code>BufferedInputStream</code> , <code>BufferedOutputStream</code> , <code>BufferedReader</code> , and <code>BufferedWriter</code> reduce the number of underlying I/O operations by buffering data.                                                                                                                                                                                                                                                                                        |
| <b>Simple Java example</b>                          | <pre> public class IOStreams {     public static void main(String[] args) throws         IOException {         try (InputStream in = new             FileInputStream("input.txt");             OutputStream out = new             FileOutputStream("output.txt")) {             byte[] buffer = new byte[1024];             int bytesRead;              while ((bytesRead = in.read(buffer)) != -1) {                 out.write(buffer, 0, bytesRead);             }         }     } } </pre> |
| <b>What the example does</b>                        | It reads the input file in chunks of bytes and writes those bytes to the output file. The try-with-resources construct automatically closes both streams.                                                                                                                                                                                                                                                                                                                                     |
| <b>Text example</b>                                 | <pre> try (BufferedReader reader = new BufferedReader(new     FileReader("input.txt"))) {     String line;     while ((line = reader.readLine()) != null) {         System.out.println(line);     } } </pre>                                                                                                                                                                                                                                                                                  |
| <b>Binary vs text</b>                               | Use <b>byte streams</b> for binary data such as images, PDFs, ZIP files,                                                                                                                                                                                                                                                                                                                                                                                                                      |

|                                             |                                                                                                                                                                                                                                                                                                             |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                             | and arbitrary bytes. Use <b>character streams</b> for text, taking character encoding into account.                                                                                                                                                                                                         |
| <b>Buffering</b>                            | Wrapping an underlying stream with a buffered stream can reduce expensive underlying I/O operations. Example: <code>new BufferedInputStream(new FileInputStream("data.bin"))</code> .                                                                                                                       |
| <b>Resource management</b>                  | Streams should be closed when no longer needed. Modern Java code should generally use <b>try-with-resources</b> so resources are closed automatically, even when an exception occurs.                                                                                                                       |
| <b>Encoding consideration</b>               | For text, prefer explicitly specifying the character set rather than relying on the platform default. For example, modern code can use <code>InputStreamReader</code> with <code>StandardCharsets.UTF_8</code> .                                                                                            |
| <b>Alternate feature — NIO/NIO.2</b>        | <b>java.nio / NIO.2</b> provides newer APIs such as <code>Path</code> , <code>Files</code> , <code>channels</code> , <code>buffers</code> , and filesystem utilities. For many modern file operations, <code>Files</code> and <code>Path</code> are more convenient than the original stream-oriented APIs. |
| <b>Alternate feature — Channels</b>         | <b>Channels</b> such as <code>FileChannel</code> provide a different I/O abstraction that can support buffers, file positioning, memory mapping, and other advanced operations.                                                                                                                             |
| <b>Alternate feature — Files API</b>        | <code>Files.readString()</code> , <code>Files.writeString()</code> , <code>Files.readAllBytes()</code> , and related methods provide concise higher-level filesystem operations when the complete data can reasonably be handled in memory.                                                                 |
| <b>Alternate feature — Asynchronous I/O</b> | NIO.2 provides asynchronous channels such as <code>AsynchronousFileChannel</code> for applications that need asynchronous I/O operations rather than traditional blocking streams.                                                                                                                          |
| <b>Alternate feature — Scanner</b>          | <code>Scanner</code> provides convenient token-oriented parsing for certain text-input scenarios, although it is not generally a replacement for streams in high-performance I/O.                                                                                                                           |
| <b>Alternate feature — PrintWriter</b>      | <code>PrintWriter</code> provides convenient formatted character output and can be preferable when the requirement is human-readable text rather than raw byte output.                                                                                                                                      |
| <b>I/O Streams vs NIO</b>                   | <b>Streams:</b> traditional sequential I/O abstraction. <b>NIO:</b> provides buffers/channels and more advanced I/O capabilities, including non-blocking and asynchronous APIs.                                                                                                                             |
| <b>I/O Streams vs Files</b>                 | <b>Streams:</b> useful when processing data incrementally. <b>Files:</b> convenient for high-level operations such as reading/writing complete files or manipulating filesystem metadata.                                                                                                                   |

|                                |                                                                                                                                                                                                                                 |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I/O Streams vs Channels</b> | <b>Streams:</b> straightforward sequential access. <b>Channels:</b> provide operations such as positioning and buffer-based I/O and are useful for more advanced file/network scenarios.                                        |
| <b>Best practice</b>           | Use <b>try-with-resources</b> , choose byte vs character streams according to the data, specify text encodings explicitly, buffer when appropriate, and prefer modern Path/Files/NIO APIs when they provide a clearer solution. |

| Aspect            | Networking Sockets                                                                                                                                                                                                                                                                                                       |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java Version      | Java 1.0                                                                                                                                                                                                                                                                                                                 |
| Year              | 1996                                                                                                                                                                                                                                                                                                                     |
| Feature           | Networking Sockets                                                                                                                                                                                                                                                                                                       |
| UniqueClassName   | NetworkSockets                                                                                                                                                                                                                                                                                                           |
| Definition        | <b>Networking sockets</b> provide a programming interface for communication between applications over a network. Java 1.0 introduced socket APIs such as Socket for TCP client communication and ServerSocket for accepting TCP connections, along with UDP support through DatagramSocket.                              |
| Main purpose      | To enable applications to <b>send and receive data over a network</b> , such as between a client and server.                                                                                                                                                                                                             |
| Main socket types | <b>TCP:</b> Socket and ServerSocket — connection-oriented, reliable, ordered byte-stream communication. <b>UDP:</b> DatagramSocket — datagram-based communication without TCP's connection-oriented reliability guarantees.                                                                                              |
| TCP client        | <pre>Socket socket = new Socket("example.com", 8080); InputStream in = socket.getInputStream(); OutputStream out = socket.getOutputStream();</pre>                                                                                                                                                                       |
| TCP server        | <pre>ServerSocket server = new ServerSocket(8080); Socket client = server.accept();</pre>                                                                                                                                                                                                                                |
| Advantages        | <ol style="list-style-type: none"> <li>1. Enables direct client-server communication.</li> <li>2. Supports TCP's reliable, ordered byte-stream communication.</li> <li>3. Supports UDP for low-overhead datagram communication.</li> <li>4. Provides relatively low-level control over network communication.</li> </ol> |



|                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                         | <p>5. Works across different machines and operating systems using standard network protocols.</p> <p>6. Integrates naturally with Java I/O streams.</p> <p>7. Suitable for custom network protocols and specialized services.</p> <p>8. Well-established and widely supported.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Disadvantages</b>                                    | <p>1. Low-level socket programming requires significant protocol and connection-management code.</p> <p>2. Blocking socket operations can occupy platform threads while waiting for network activity.</p> <p>3. Applications must handle connection failures, timeouts, partial reads/writes, and disconnections correctly.</p> <p>4. TCP provides a byte stream, not application-level messages, so message framing must be designed by the application.</p> <p>5. Security such as encryption and authentication must be deliberately provided; a plain socket is not automatically secure.</p> <p>6. Large numbers of blocking connections can require substantial thread resources if each connection uses a dedicated platform thread.</p> <p>7. Custom protocols can be harder to maintain than standardized application protocols.</p> |
| <b>When to use</b>                                      | Use sockets when you need <b>direct, low-level network communication</b> , especially for custom TCP/UDP protocols, specialized client-server applications, local network services, multiplayer/network applications, or integration with systems that expose a socket protocol.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>When not to use</b>                                  | Avoid raw sockets when a standardized higher-level protocol/API already solves the problem more appropriately—for example, HTTP APIs for web services, WebSocket for browser-oriented bidirectional communication, or messaging systems for asynchronous distributed communication.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Real-time use case 1 — Client-server application</b> | A desktop client can connect to a backend server using TCP sockets to send requests and receive responses.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Real-time use case 2 — Multiplayer games</b>         | Games can use UDP or TCP-based networking depending on whether low latency or reliable ordered delivery is more important for a particular type of data.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Real-time use case 3 — Chat application</b>          | A custom chat system can maintain TCP connections between clients and a server for reliable message delivery. Modern applications often use WebSocket or messaging infrastructure instead.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Real-time use case 4 — IoT communication</b>         | Embedded devices can communicate with a custom server over TCP or UDP when the device protocol is specifically designed around sockets.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

|                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Real-time use case 5 — Internal services</b> | Private infrastructure can use TCP sockets for custom service-to-service protocols when HTTP or another standard protocol is unnecessary or unsuitable.                                                                                                                                                                                                                                                                                                                                                                                      |
| <b>Real-time use case 6 — Network utilities</b> | Tools such as custom monitoring agents, diagnostic programs, and network services can communicate directly using sockets.                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Simple TCP server example</b>                | <pre> public class NetworkSockets {     public static void main(String[] args) throws         IOException {         try (ServerSocket server = new ServerSocket(5000))         {             try (Socket client = server.accept();                 BufferedReader in = new BufferedReader(                     new                         InputStreamReader(client.getInputStream()))) {                 String message = in.readLine();                 System.out.println("Received: " + message);             }         }     } } </pre> |
| <b>Simple TCP client example</b>                | <pre> try (Socket socket = new Socket("localhost", 5000);     PrintWriter out = new         PrintWriter(socket.getOutputStream(), true)) {     out.println("Hello Server"); } </pre>                                                                                                                                                                                                                                                                                                                                                         |
| <b>How the example works</b>                    | The server creates a <code>ServerSocket</code> and waits with <code>accept()</code> . A client creates a <code>Socket</code> and connects to the server. The client writes data through the socket's output stream, and the server reads it through its input stream.                                                                                                                                                                                                                                                                        |
| <b>TCP characteristic</b>                       | TCP provides a reliable, ordered <b>byte stream</b> . It does not preserve application-level message boundaries. For example, one <code>println()</code> on one side does not inherently correspond to one <code>read()</code> on the other side; the application must define framing such as lines, lengths, or delimiters.                                                                                                                                                                                                                 |
| <b>UDP example</b>                              | <pre> DatagramSocket socket = new DatagramSocket();byte[] data =     "Hello".getBytes(StandardCharsets.UTF_8);DatagramPacket     packet = new DatagramPacket(`data, data.length,         InetAddress.getLoopbackAddress(), 5000);socket.send(packet); </pre>                                                                                                                                                                                                                                                                                 |
| <b>TCP vs UDP</b>                               | <b>TCP:</b> connection-oriented, reliable, ordered byte stream. <b>UDP:</b> connectionless datagrams with no built-in guarantee of delivery, ordering, or duplicate prevention.                                                                                                                                                                                                                                                                                                                                                              |

|                                                  |                                                                                                                                                                                                                            |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Important security consideration</b>          | Plain TCP/UDP sockets do <b>not automatically provide encryption or authentication</b> . For sensitive communication, use an appropriate secure protocol such as TLS (SSLSocket) or a secure application-level protocol.   |
| <b>Alternate feature — HTTP/HTTPS</b>            | <b>HTTP/HTTPS</b> is usually preferable for web APIs and request-response communication because it provides standardized semantics, tooling, authentication mechanisms, proxies, caching, and extensive ecosystem support. |
| <b>Alternate feature — WebSocket</b>             | <b>WebSocket</b> provides persistent, bidirectional communication and is particularly useful for browser-to-server real-time applications such as chat, notifications, and live dashboards.                                |
| <b>Alternate feature — RMI</b>                   | <b>Java RMI</b> provides higher-level remote-object communication for Java-to-Java distributed applications, although it is less common in modern distributed architectures.                                               |
| <b>Alternate feature — Messaging systems</b>     | <b>Message queues/brokers</b> are preferable when communication should be asynchronous, buffered, decoupled, or resilient to temporary consumer unavailability.                                                            |
| <b>Alternate feature — NIO channels</b>          | <b>Java NIO channels</b> such as SocketChannel and ServerSocketChannel provide a more scalable, buffer-oriented networking model and support non-blocking I/O.                                                             |
| <b>Alternate feature — Asynchronous channels</b> | <b>NIO.2 asynchronous channels</b> can perform network operations asynchronously, reducing the need to dedicate a platform thread to every blocking network operation.                                                     |
| <b>Sockets vs HTTP</b>                           | <b>Sockets:</b> low-level, flexible, custom protocols. <b>HTTP:</b> standardized application-level protocol and generally easier for web/API communication.                                                                |
| <b>Sockets vs WebSocket</b>                      | <b>Raw socket:</b> greater control and lower-level protocol design. <b>WebSocket:</b> standardized bidirectional protocol, especially convenient for browser applications.                                                 |
| <b>Sockets vs Messaging</b>                      | <b>Sockets:</b> direct communication between endpoints. <b>Messaging:</b> decouples producers and consumers and can provide buffering, retries, and delivery semantics depending on the broker.                            |
| <b>Sockets vs NIO</b>                            | <b>Traditional Socket:</b> commonly blocking and stream-oriented. <b>NIO:</b> channel/buffer-based and can support non-blocking multiplexed I/O.                                                                           |
| <b>Best practice</b>                             | Use raw sockets when their low-level control is genuinely required. Set appropriate <b>timeouts</b> , handle partial reads/writes and disconnections, define explicit message framing, close                               |

|  |                                                                                                                       |
|--|-----------------------------------------------------------------------------------------------------------------------|
|  | resources with try-with-resources, and use TLS or another appropriate security mechanism for sensitive communication. |
|--|-----------------------------------------------------------------------------------------------------------------------|